# Лабораторная работа № 2
# Введение в алгоритмическую и высокоуровневую оптимизацию на примере задачи умножения матриц

**Цель работы:** познакомиться с базовыми принципами выполнения алгоритмической и высокоуровневой оптимизации программного кода на примере задачи умножения квадратных матриц.

Алгоритмической оптимизацией называется совокупность приемов, позволяющая решить ту же самую задачу с использованием более быстрых алгоритмов (реже, алгоритмов, реализация которых требует меньших затрат памяти, регистров и т.п.). Высокоуровневой оптимизацией называется оптимизация на уровне структурных элементов программы (подпрограмм, циклов, линейных участков). Обычно целью программной оптимизации является снижение времени, необходимого программе для решения поставленной задачи. В данной работе указанные виды оптимизации обсуждаются в контексте задачи умножения двух квадратных матриц размера $N \times N$.

*Классическое умножение матриц* реализуется в соответствии с известной формулой

$$c_{ij} = \sum_{k=1}^{N} a_{ik} b_{kj} \ \forall i, j = \overline{1, N} \tag{1}$$

следующей программой (тип элементов матрицы – `float`):

```
for (size_t i=0; i<N; i++)
        for (size_t j=0; j<N; j++)
        {
                float s = 0.0f;

                for (size_t k=0; k<N; k++)
                        s += A[i][k]*B[k][j];

                C[i][j] = s;
        }
```

Данная реализация довольно проста, однако неэффективна, когда размер матриц превышает объем кэш-памяти процессора. При выполнении умножения обход элементов матрицы $A$ производится подряд $\left(a_{i,1}, a_{i,2}, ..., a_{i,N}, a_{i+1,1}, a_{i+1,2}, ..., a_{i+1,N}, ...\right)$, при этом эффективно работает кэш-память, предзагрузка данных реализуется с использованием механизма hardware prefetch и не требует внимания со стороны программиста. Обход элементов матрицы $B$ напротив производится прыжками через $4N$ байт $\left(b_{1,j}, b_{2,j}, ..., b_{N,j}, b_{1,j+1}, b_{2,j+1}, ..., b_{N,j+1}, ...\right)$, что мешает эффективной работе кэш-памяти и ограничивает быстродействие, т.к. при каждом обращении к элементу $b_{kj}$ матрицы $B$ из памяти в кэш-память производится передача порции данных (англ. chunk) размером 64–128 байт (зависит от типа процессора и физически реализованной разрядности шин между памятью и различными уровнями кэшей процессора), из которой используется только 4 байта – `sizeof(float)`.

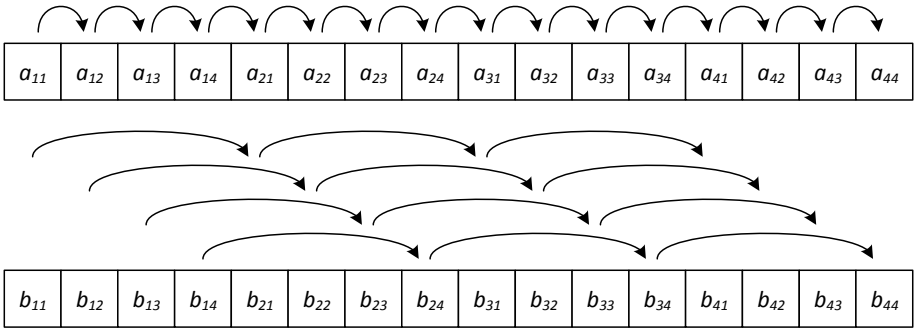


Рис. 1. Паттерн обращения к памяти при реализации классического алгоритма умножения матриц

Простейшим способом обойти данное ограничение является транспонирование матрицы $B$ с последующим выполнением умножения по той же формуле (1). В данном случае при обращениях к коэффициентам обеих матриц обход памяти производится подряд удобным для процессора способом. Однако недостатками данной реализации являются существенные затраты времени на выполнение транспонирования матрицы $B$ (в особенности если ее размер в байтах превышает емкость кэш памяти процессора) и размещение данных в памяти (а не в кэш памяти) в случае, если суммарный размер перемножаемых матриц $A$ и $B$ превышает размер кэша.

Еще один способ, именуемый *буферизованным умножением*, реализуется путем поочередного сохранения столбцов матрицы $B$ во временном одномерном массиве. Чтобы обойти указанное выше ограничение на неоптимальный порядок обхода элементов матрицы $B$, можно ввести временный массив

```
float tmp[N];
```

и занести в него элементы $j$-го столбца матрицы $B$. При этом при выполнении умножения $i$-х строк матрицы $A$, $i=\overline{1,N}$, на элементы $j$-го столбца кэш-память процессора работает максимально эффективно, однако при этом появляются накладные расходы, связанные с необходимостью копирования элементов столбца (подсказка: при практической реализации данного способа порядок выполнения циклов $(i, j, k)$ меняется на $(j, i, k)$).

Еще одним способом умножения матриц является *блочное умножение*. Можно заметить, что результат умножения может быть получен путем независимого вычисления подматриц $C'_{xy}$ матрицы $C$ размером $S \times S$ элементов, $S=\dfrac{N}{k}$, $k$ – обычно некоторое целое число (см. рис. 2).

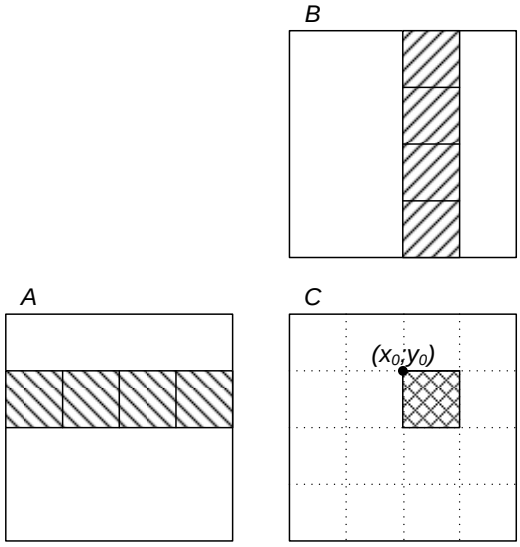


Рис. 2. Иллюстрация к блочному умножению матриц

При этом для вычисления значений, входящих в состав текущей подматрицы $C'_{xy}$, необходимы не все значения матриц $A$ и $B$, а лишь узкие «полосы» значений размером $4 \times S \times N$ байт каждая, отмеченные на рисунке штриховкой. Дополнительно снизить требования к объему используемой кэш-памяти можно, заметив, что результирующие значения могут быть получены как суммы результирующих значений умножения подматриц. Для приведенного примера

$$C'_{23} = A'_{21} \times B'_{13} + A'_{22} \times B'_{23} + A'_{23} \times B'_{33} + A'_{24} \times B'_{43} .$$

Для вычисления каждого из произведений $A'_{xz} \times B'_{zy}$ необходимы лишь значения подматриц $A'_{xz}$ и $B'_{zy}$ объемом $4 \times S^2$ байт каждая. Текущие обрабатываемые подматрицы можно скопировать в соответствующие временные массивы, подматрицу $B'_{zy}$ при данном копировании можно транспонировать «на лету».

При выполнении умножения основным действием в цикле является накапливающее сложение произведений элементов матриц:

```
for (int k=0; k<N; k++)
        s += A[i][k]*B[k][j];
```

Оно состоит из двух обращений к памяти, умножения и сложения. Обращения к памяти при чтении значений $a_{ik}$ и $b_{kj}$ теоретически могут производиться параллельно, в то время как умножение и последующее сложение имеют последовательные зависимости по данным и могут выполняться только последовательно, что уменьшает параллелизм на уровне команд (англ. Instruction Level Parallelism, сокр. ILP). Для его увеличения необходимо осуществить раскрутку цикла (англ. Loop Unrolling) на некоторое количество итераций $M$. Пример раскрутки цикла на 2 итерации приведен ниже:

```
for (int k=0; k<N; k+=2)
{
        s1 += A[i][k]*B[k][j];
        s2 += A[i][k+1]*B[k+1][j];
}
```

```
s = s1 + s2;
```

Раскрутка на разумное число итераций приводит к повышению ILP, появлению более длинного линейного (без команд условных переходов) участка ассемблерного кода, повышению степени загрузки исполнительных устройств (англ. Executing Units, сокр. EU) и, соответственно, к сокращению времени выполнения умножения матриц. Чрезмерная раскрутка цикла может привести к исчерпанию емкости кэша команд L1i и, как следствие, к существенному увеличению времени обработки.

**Задание.** В соответствии с индивидуальным вариантом реализовать следующие варианты кода для умножения матриц размером до 8192×8192 (*N* задается):

1. Классическое умножение матриц.
2. Классическое умножение матриц с предварительным транспонированием матрицы *B*.
3. Умножение матриц с использованием буферизации столбца матрицы *B*.
4. Блочное умножение матриц (размер блока *S* задается как параметр).

Сравнить быстродействие классического умножения матриц в конфигурациях запуска «Debug» без оптимизации компилятора и «Release» с оптимизацией. Сделать выводы о получаемом выигрыше в скорости обработки.

Убедиться в том, что результаты умножения для всех алгоритмов совпадают. Для этого разработать отдельную подпрограмму, сравнивающую полученные матрицы *C* для различных алгоритмов умножения.

Измерить время *t* выполнения для каждого алгоритма. Вычислить реальную производительность используемой вычислительной системы в рассматриваемой задаче умножения матриц по формуле $P_{real} = \frac{V}{t}$ (в GFLOP/s), где $2N^3$ – объем FP-вычислений в рассматриваемой задаче ($N^3$ сложений и $N^3$ умножений).

Для алгоритма блочного умножения вычислить $P_{real}$ в зависимости от размера блока *S* (размер блока взять равным степеням двойки: 1, 2, 4, 8, …), построить графики зависимости $P_{real}$ от *S* для фиксированного *N*. Сделать вывод об оптимальном размере блока $S^*$.

Произвести раскрутку внутренних циклов для умножения с буферизацией столбца и блочного умножения на различное число итераций *M* (выбирать равным степеням двойки (1, 2, 4, 8, 16), следить за размером блока при блочном умножении). Оценить его влияние на время выполнения умножения, построить графики зависимости $P_{real}$ от *M*, сделать вывод об оптимальном значении степени раскрутки $M^*$ для каждой из реализаций (для каждого значения *S*).

Сравнить быстродействие наилучших реализаций (с использованием оптимальных значениями $S^*$ и $M^*$) в конфигурации запуска «Release» для различных значений *N* (например, 4, 8, 16, 32, …, 8192), сделать выводы о целесообразности применимости на практике различных алгоритмов умножения матриц

Замеры времени и оценку реальной производительности по возможности выполнить на 2–3 современных процессорах с различной микроархитектурой.

**Содержание отчета.**

1. Титульный лист.
2. Цель работы.
3. Описание условия решаемой задачи в соответствии с индивидуальным вариантом
4. Листинг программы.

5. Результаты сопоставления реализации алгоритма классического умножения матриц в конфигурациях запуска «Debug» и «Release».
6. Результаты измерения реальной производительности $P_{real}$ алгоритма умножения с транспонированием матрицы $B$ с учетом и без учета времени на выполнение транспонирования, сопоставление их между собой и с результатами алгоритма классического умножения матриц.
7. Результаты измерения реальной производительности $P_{real}$ алгоритма умножения с использованием буферизации столбца для выбранного $N$, графики зависимости $P_{real}$ от $M$, выводы об оптимальном значении $M^*$.
8. Результаты измерения реальной производительности $P_{real}$ с использованием алгоритма блочного умножения для выбранного $N$, графики зависимости $P_{real}$ от $M$ и $S$, выводы об оптимальных значениях $M^*$ и $S^*$.
9. Графики зависимости реальной производительности $P$ от размера матриц $N$ для всех 4 наилучших реализаций (выбранный алгоритм, оптимальная степень раскрутки $M$, оптимальный размер блока $S$), разработанных выше.
10. Выводы о целесообразности применимости различных алгоритмов умножения матриц на практике.